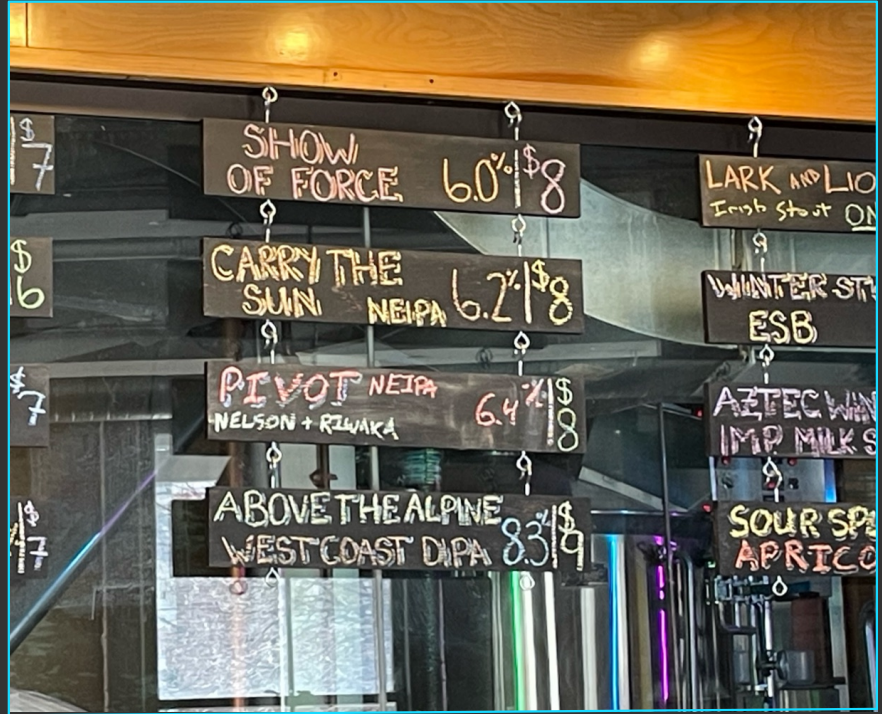


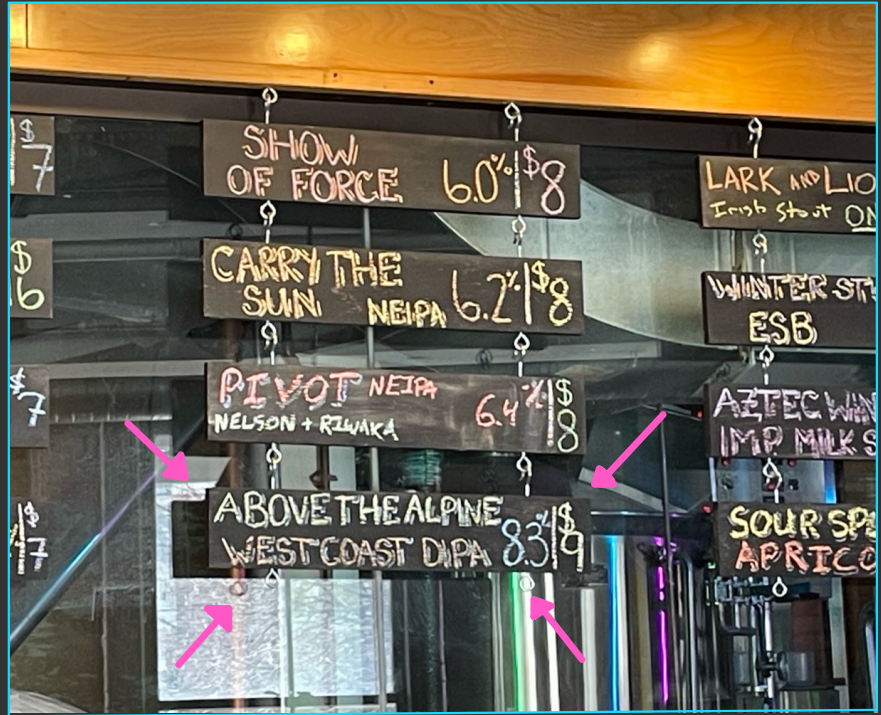
linked lists

CSCI
134

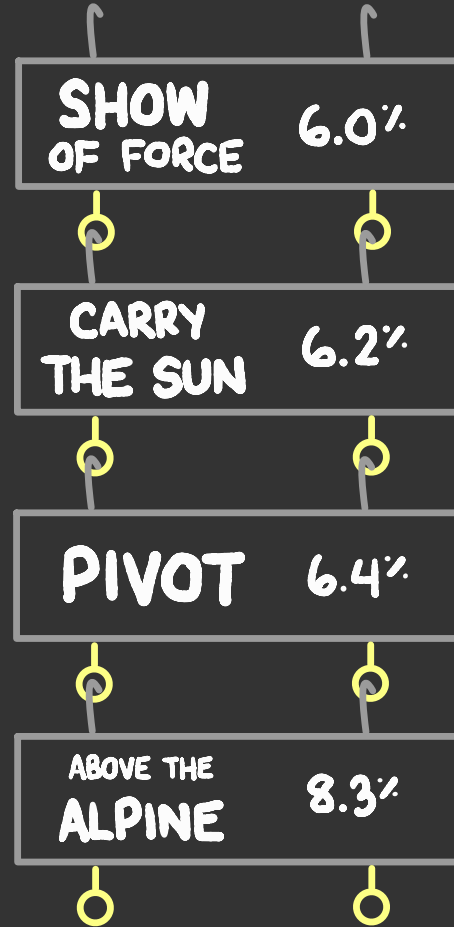
bright ideas
brewing
north adams



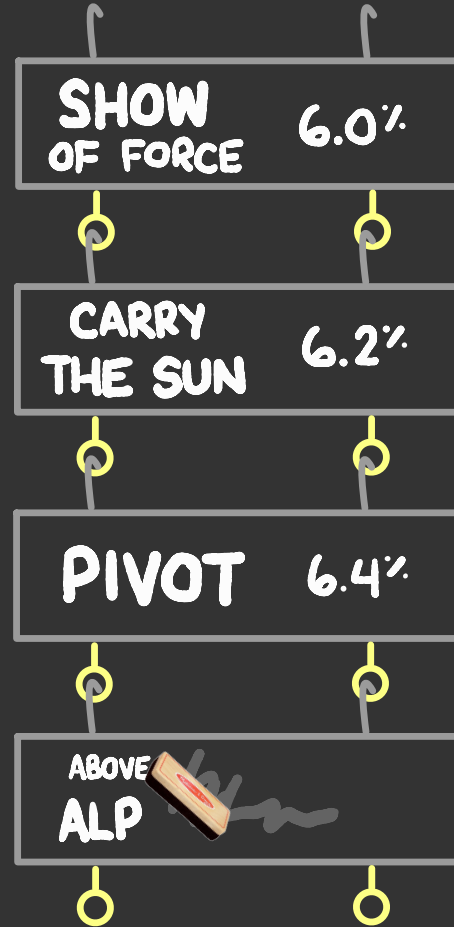
what if
above the alpine
"kicks"?



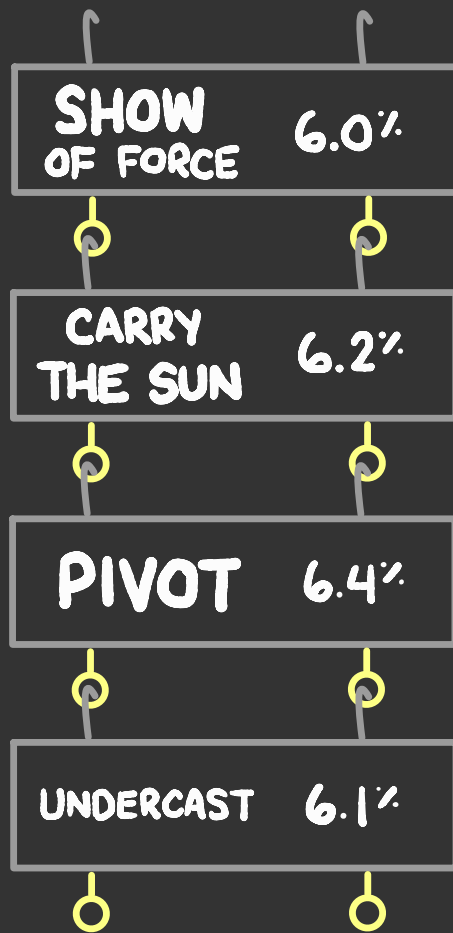
it's ok!
they have kegs of
their **flagship ipa**
in the back room



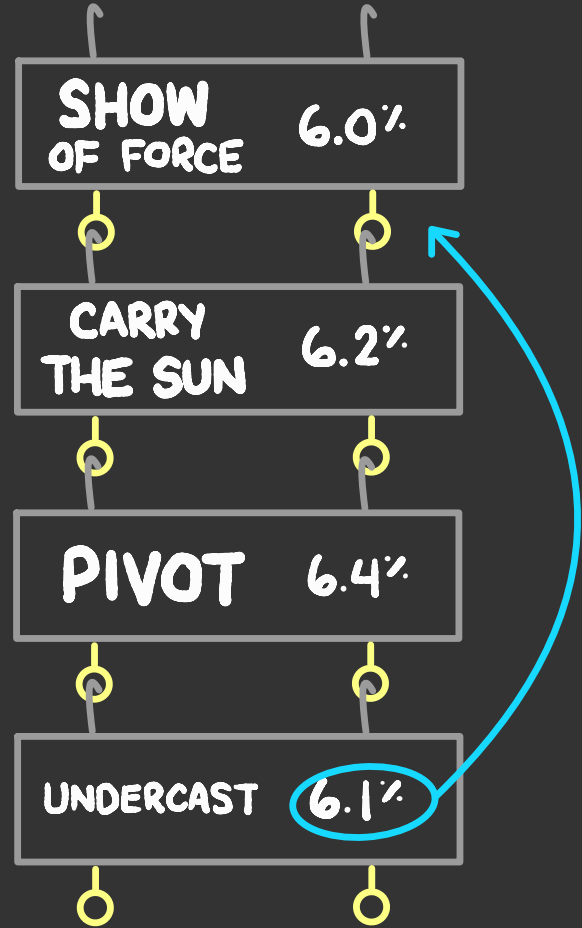
it's ok!
they have kegs of
their **flagship ipa**
in the back room



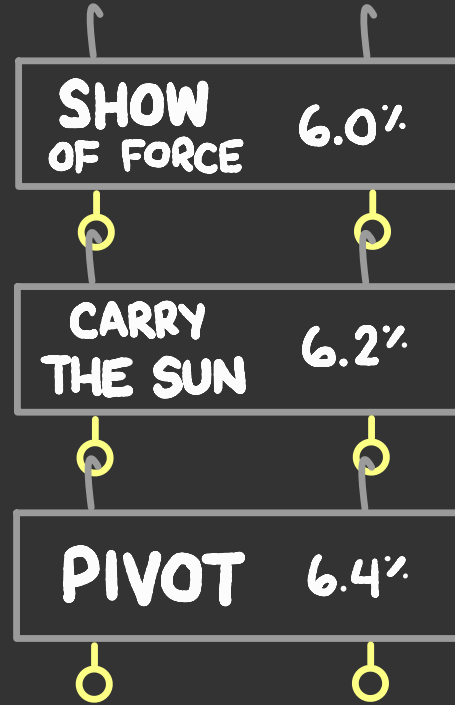
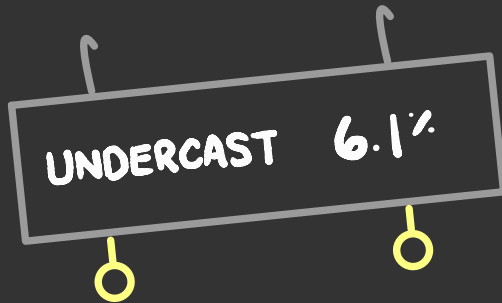
it's ok!
they have kegs of
their **flagship ipa**
in the back room



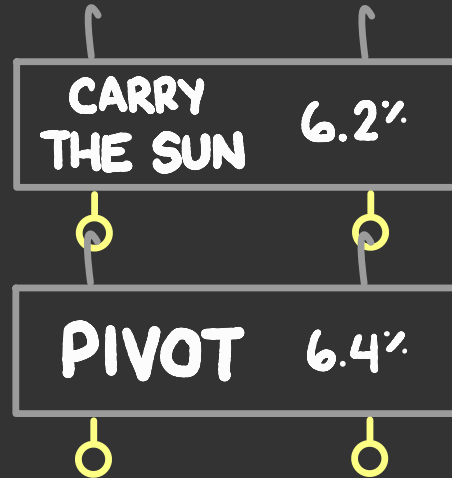
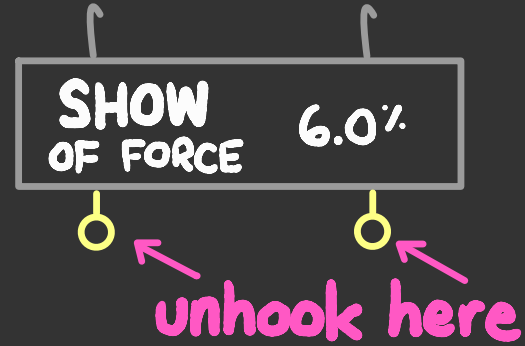
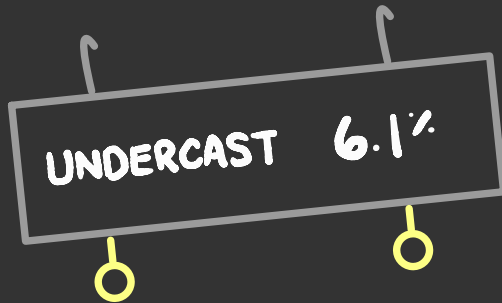
but now the list is
no longer sorted by
alcohol content



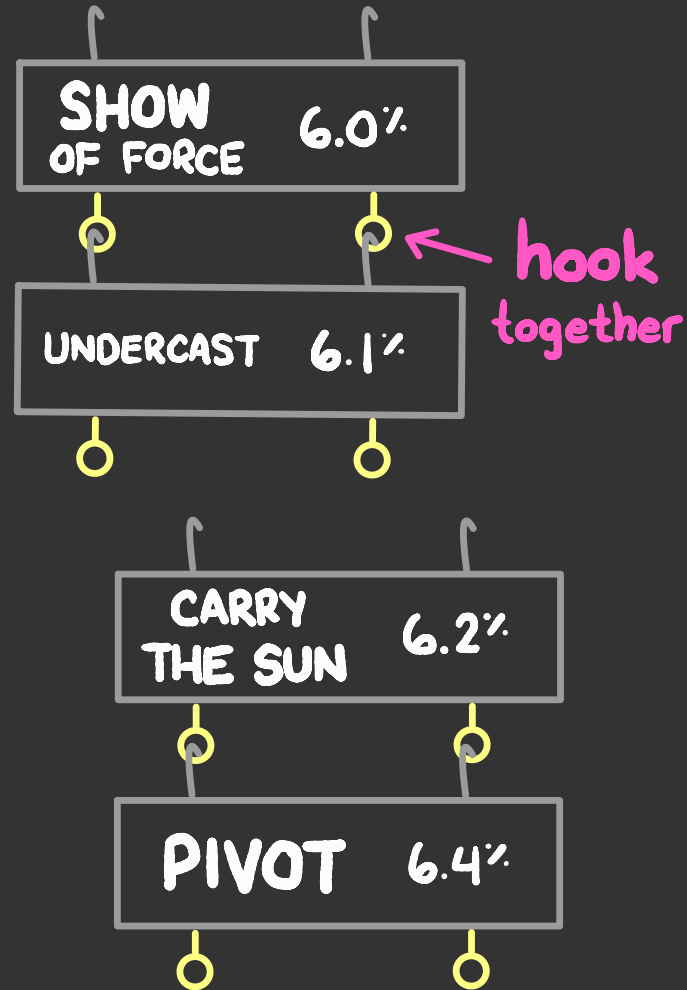
but their modular
system makes it
easy to **insert** the
new menu item in
its proper place



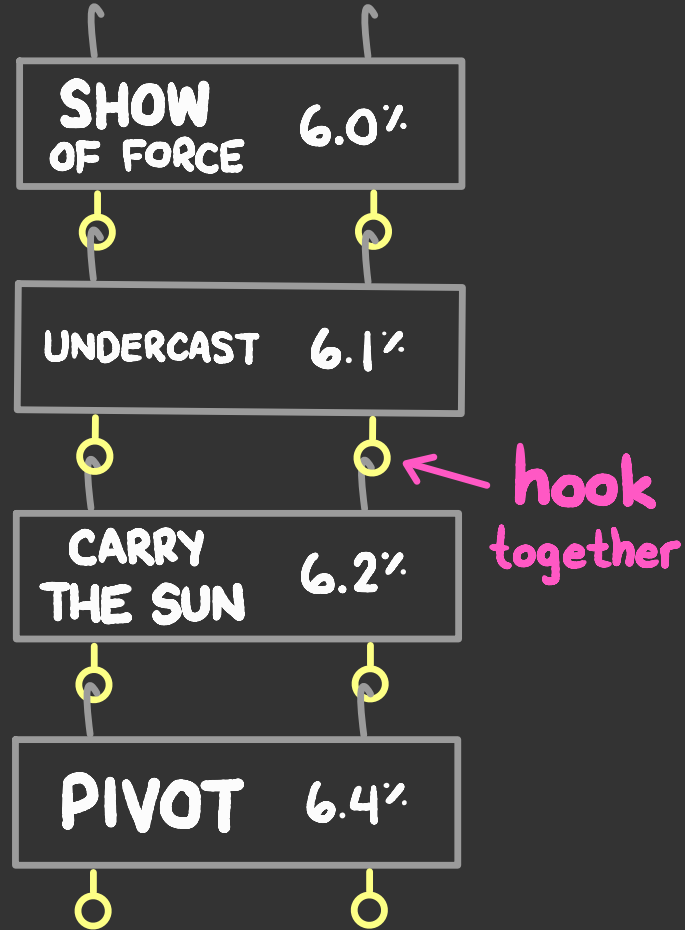
but their modular
system makes it
easy to **insert** the
new menu item in
its proper place



but their modular system makes it easy to **insert** the new menu item in its proper place

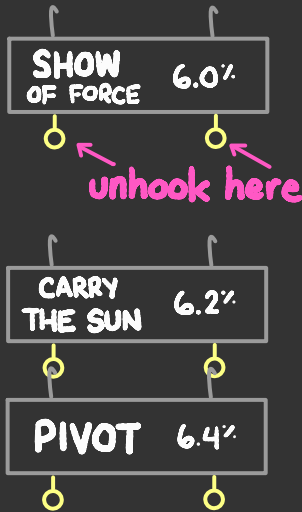


but their modular system makes it easy to **insert** the new menu item in its proper place

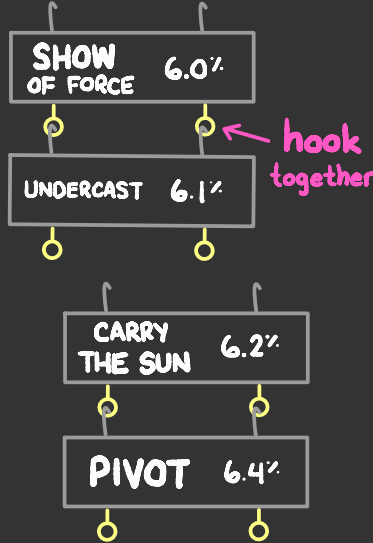


this only takes **three** operations

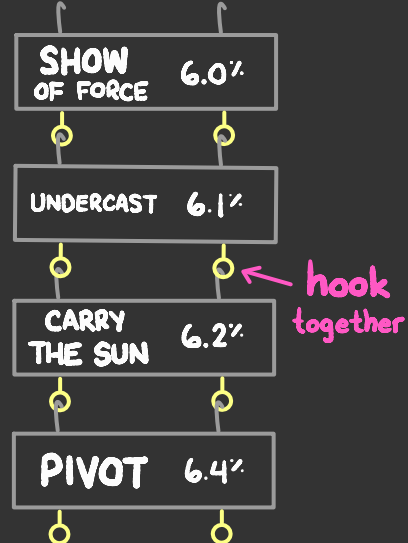
1



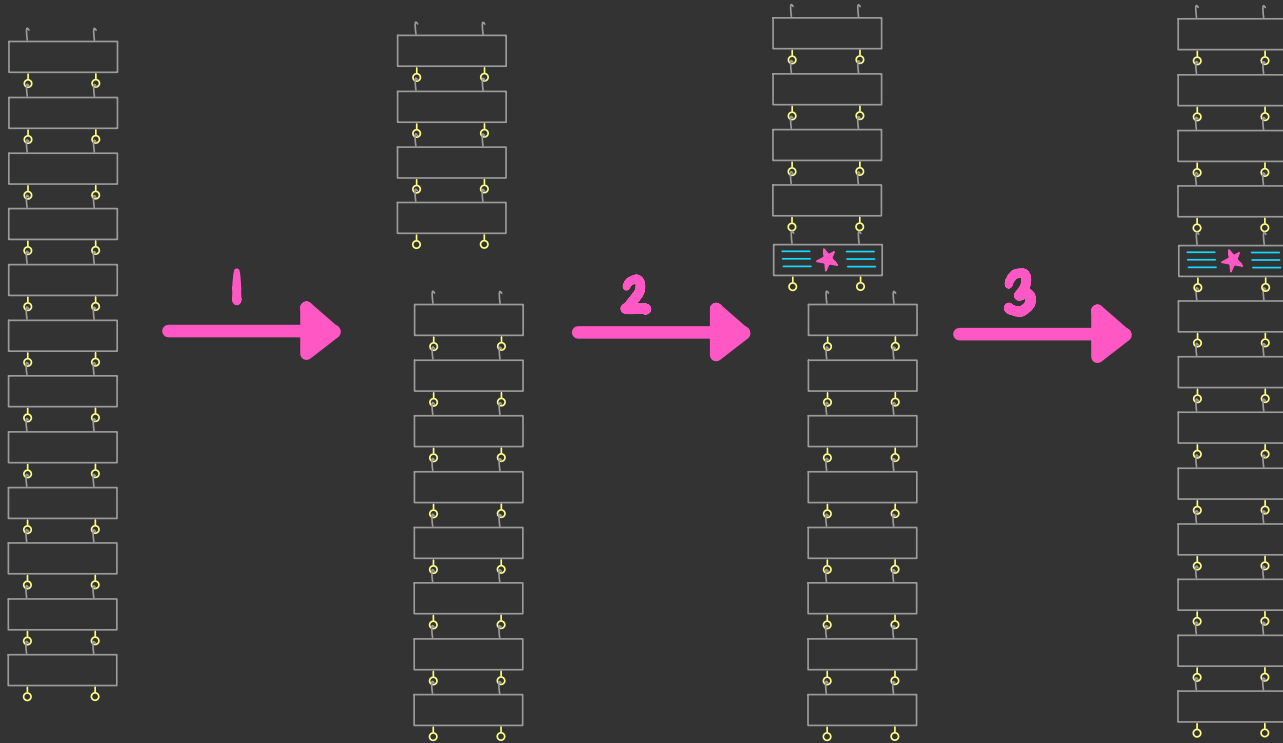
2



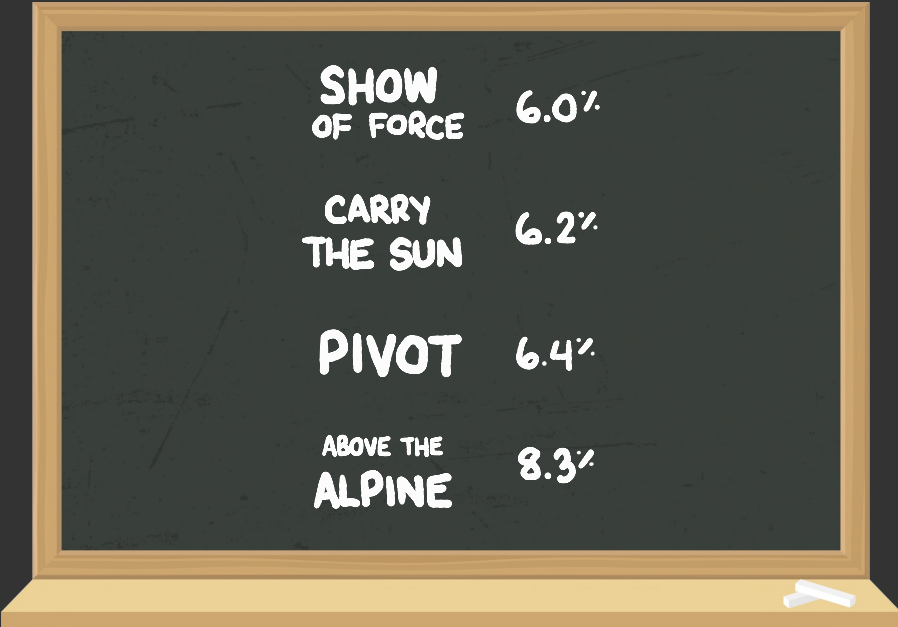
3



no matter **how long** the menu is

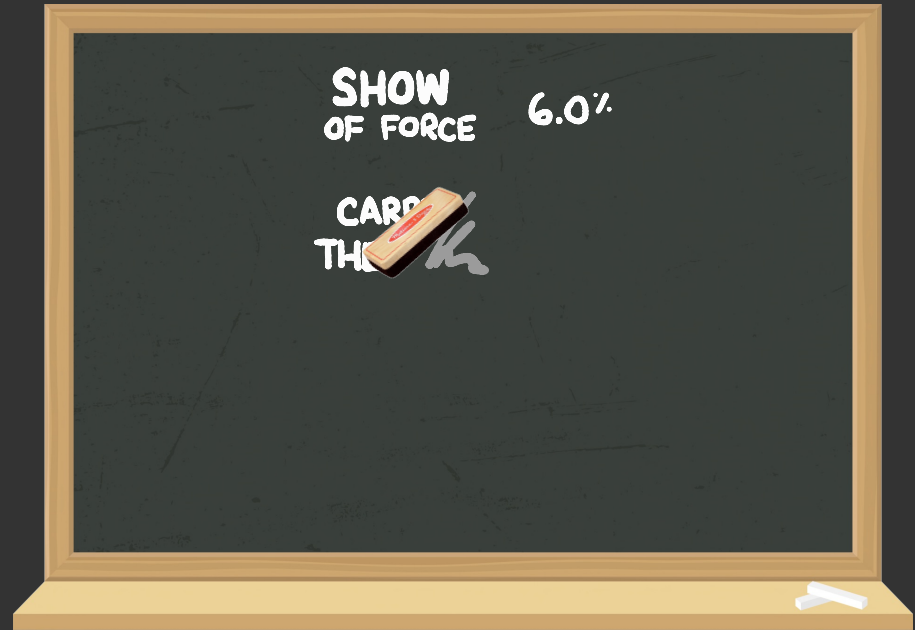


now suppose
they wrote the
menu on a **single**
chalkboard

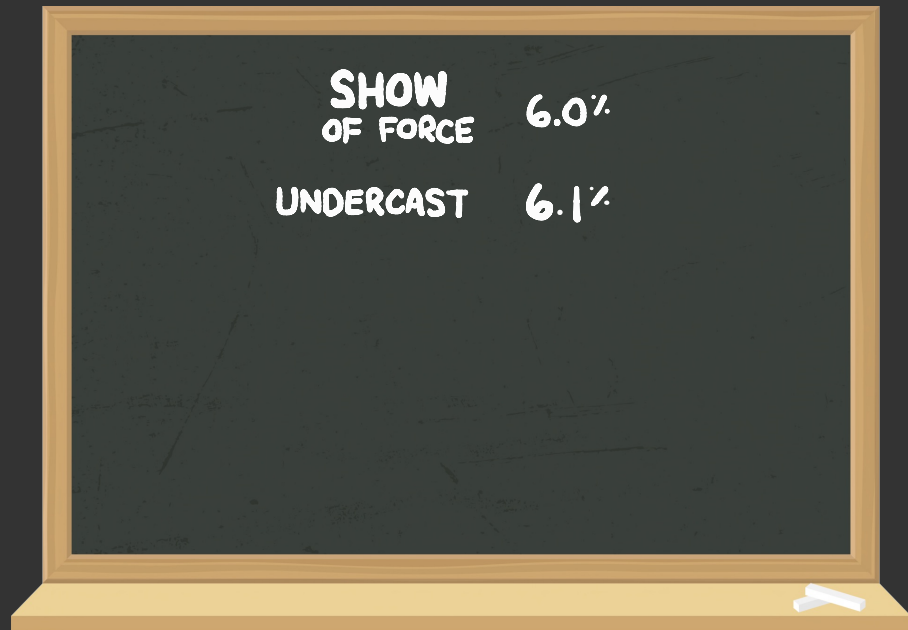


SHOW OF FORCE	6.0%
CARRY THE SUN	6.2%
PIVOT	6.4%
ABOVE THE ALPINE	8.3%

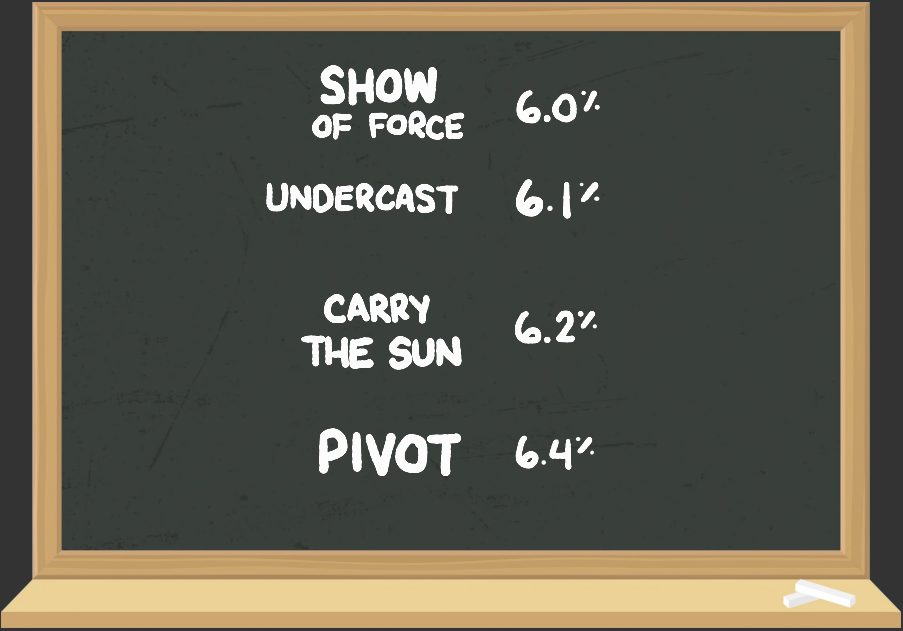
replacing a menu
item requires them
to **erase all items**
with a higher
alcohol content



then write the
new menu item



then **rewrite**
the erased items

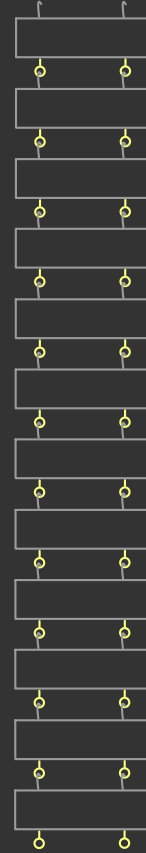


SHOW OF FORCE	6.0%
UNDERCAST	6.1%
CARRY THE SUN	6.2%
PIVOT	6.4%

in the worst case, we
might have to rewrite

every

menu item



way more
than 3

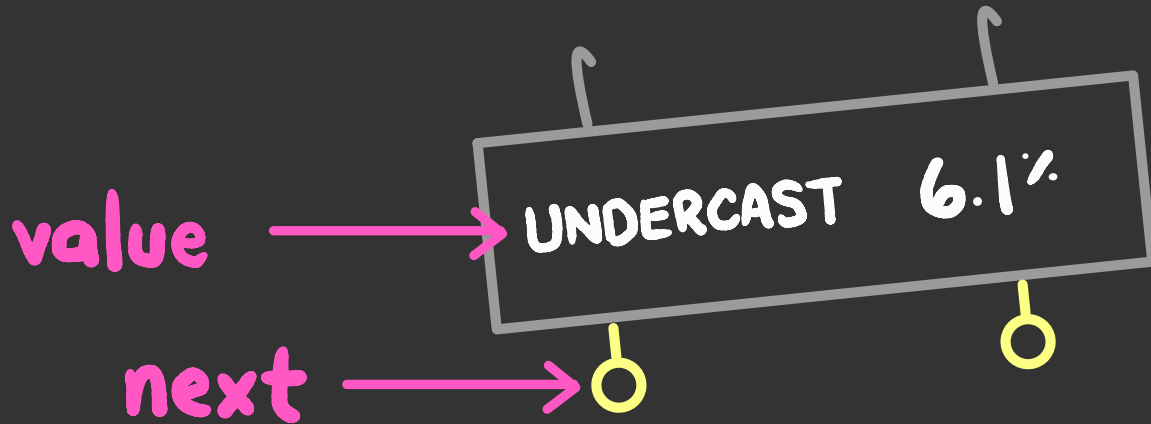
seem familiar?

```
def insert(element, ls, position):  
    """Inserts an element into a list at the specified position."""  
    return ls[:position] + [element] + ls[position:]
```

rewrites all values
up to position

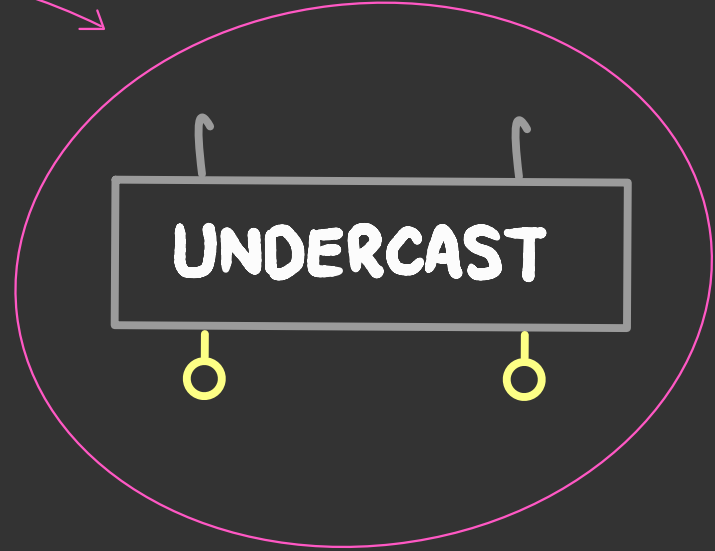
rewrites all values
from position onward

but we can emulate the
modular approach in code



```
def make_node(x, y):  
    | return [x, y]  
  
def value(node):  
    | return node[0]  
  
def next(node):  
    | return node[1]  
  
def set_value(node, val):  
    | node[0] = val  
  
def set_next(node, next_node):  
    | node[1] = next_node
```

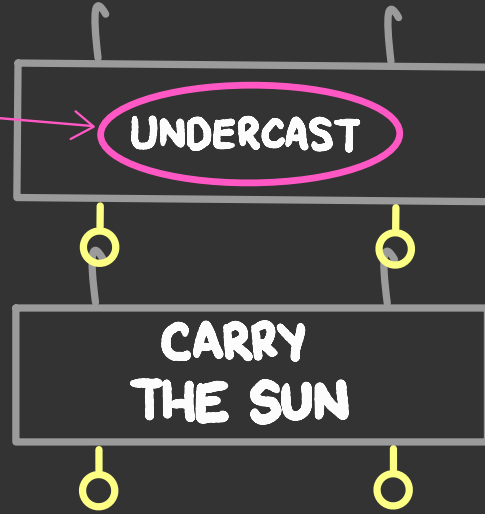
Code



"node"

```
def make_node(x, y):  
    return [x, y]  
  
def value(node):  
    return node[0]  
  
def next(node):  
    return node[1]  
  
def set_value(node, val):  
    node[0] = val  
  
def set_next(node, next_node):  
    node[1] = next_node
```

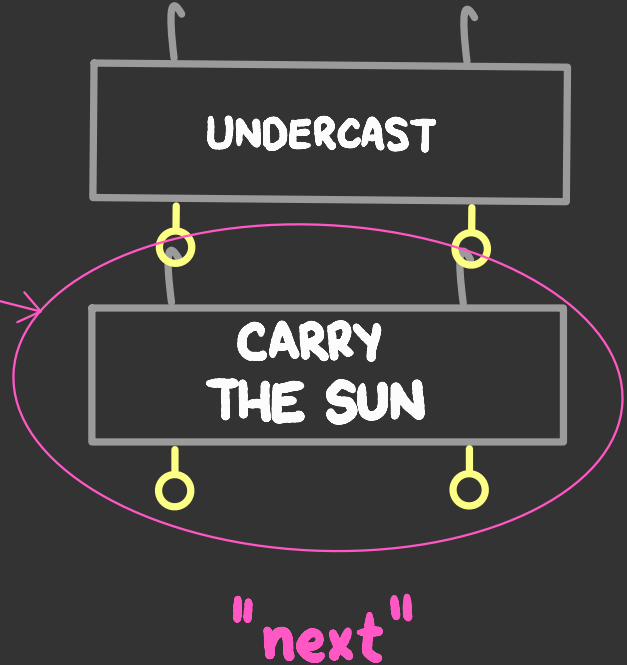
Code



"value"

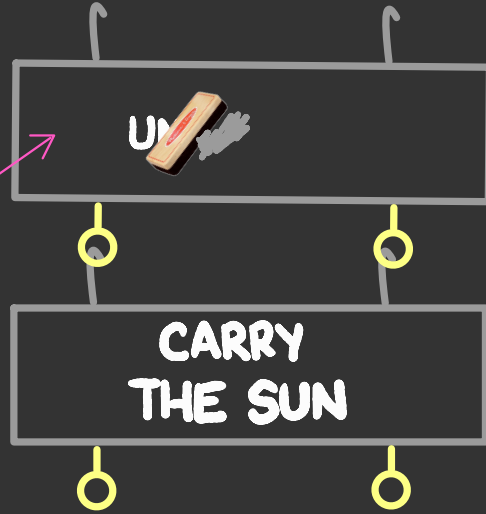
```
def make_node(x, y):  
    |   return [x, y]  
  
def value(node):  
    |   return node[0]  
  
def next(node):  
    |   return node[1]  
  
def set_value(node, val):  
    |   node[0] = val  
  
def set_next(node, next_node):  
    |   node[1] = next_node
```

Code



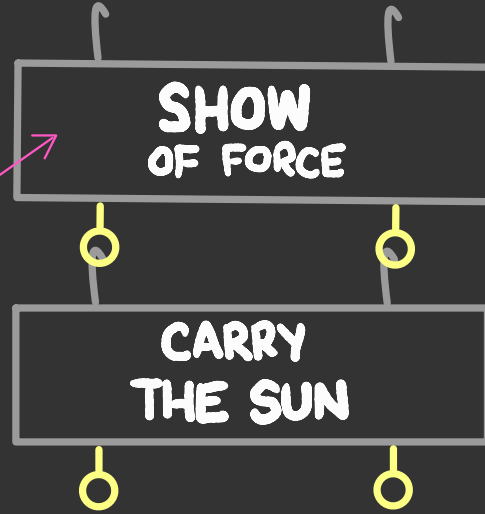
```
def make_node(x, y):  
    |   return [x, y]  
  
def value(node):  
    |   return node[0]  
  
def next(node):  
    |   return node[1]  
  
def set_value(node, val):  
    |   node[0] = val  
  
def set_next(node, next_node):  
    |   node[1] = next_node
```

Code



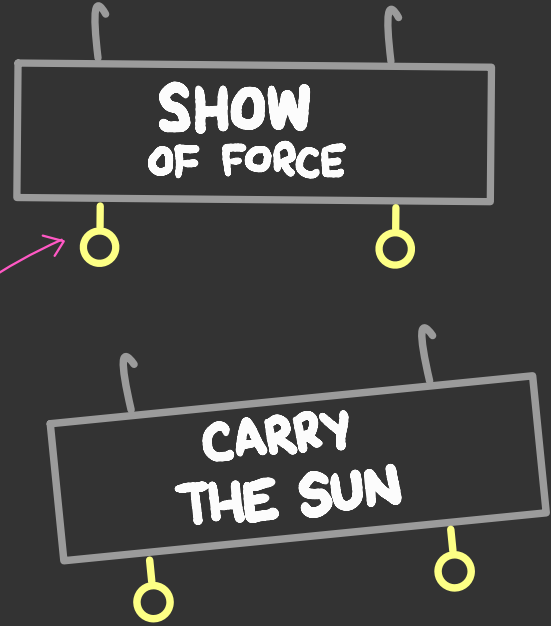
```
def make_node(x, y):  
    return [x, y]  
  
def value(node):  
    return node[0]  
  
def next(node):  
    return node[1]  
  
def set_value(node, val):  
    node[0] = val  
  
def set_next(node, next_node):  
    node[1] = next_node
```

Code



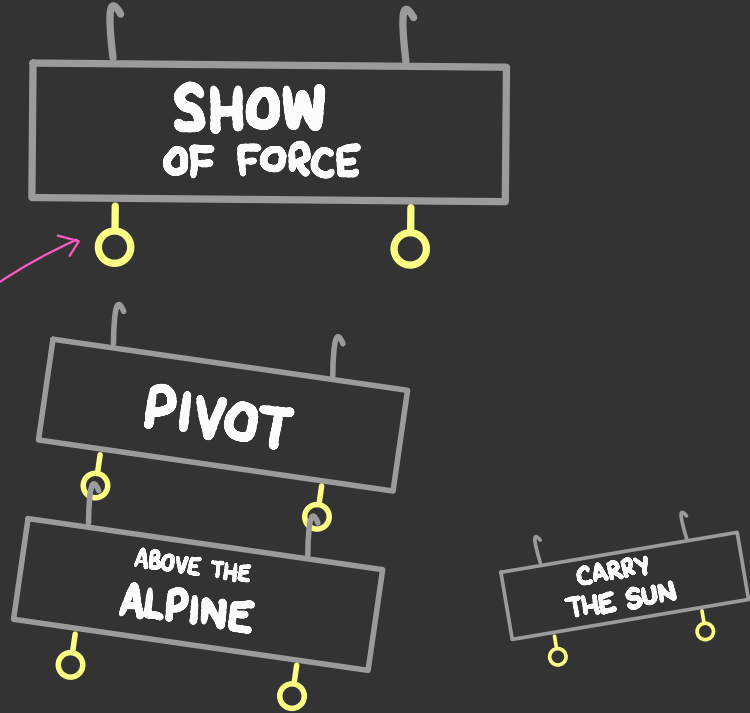
```
def make_node(x, y):  
    return [x, y]  
  
def value(node):  
    return node[0]  
  
def next(node):  
    return node[1]  
  
def set_value(node, val):  
    node[0] = val  
  
def set_next(node, next_node):  
    node[1] = next_node
```

Code



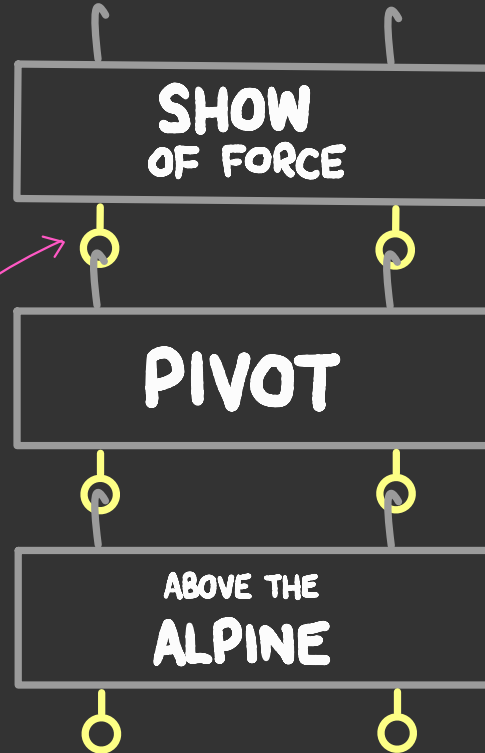
```
def make_node(x, y):  
    return [x, y]  
  
def value(node):  
    return node[0]  
  
def next(node):  
    return node[1]  
  
def set_value(node, val):  
    node[0] = val  
  
def set_next(node, next_node):  
    node[1] = next_node
```

Code



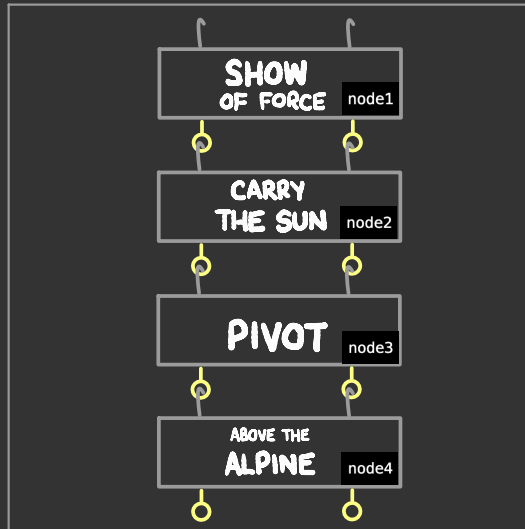

```
def make_node(x, y):  
    return [x, y]  
  
def value(node):  
    return node[0]  
  
def next(node):  
    return node[1]  
  
def set_value(node, val):  
    node[0] = val  
  
def set_next(node, next_node):  
    node[1] = next_node
```

Code



a menu update at
bright ideas
brewing

```
In [1]: from linkedlist import *  
  
In [2]: node4 = make_node("ABOVE THE ALPINE", None)  
  
In [3]: node3 = make_node("PIVOT", node4)  
  
In [4]: node2 = make_node("CARRY THE SUN", node3)  
  
In [5]: node1 = make_node("SHOW OF FORCE", node2)  
  
In [6]: node1  
Out[6]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['ABOVE THE ALPINE', None]]]]
```



```
def make_node(x, y):  
    |     return [x, y]  
  
def value(node):  
    |     return node[0]  
  
def next(node):  
    |     return node[1]  
  
def set_value(node, val):  
    |     node[0] = val  
  
def set_next(node, next_node):  
    |     node[1] = next_node
```

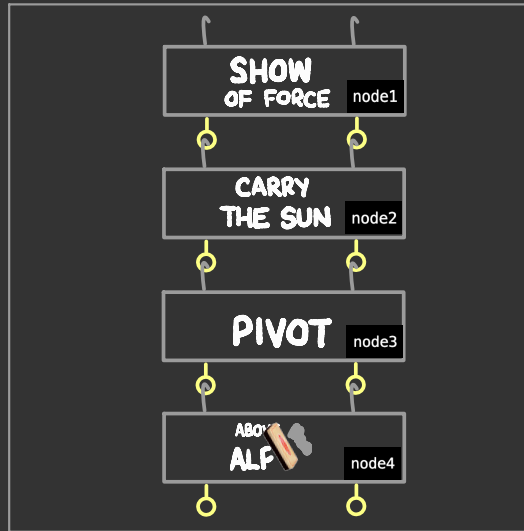
Code

a menu update at
bright ideas
brewing

```
In [6]: node1
Out[6]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['ABOVE THE ALPINE', None]]]]

In [7]: set_value(node4, "UNDERCAST")

In [8]: node1
Out[8]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['UNDERCAST', None]]]]
```



```
def make_node(x, y):
    | return [x, y]

def value(node):
    | return node[0]

def next(node):
    | return node[1]

def set_value(node, val):
    | node[0] = val

def set_next(node, next_node):
    | node[1] = next_node
```

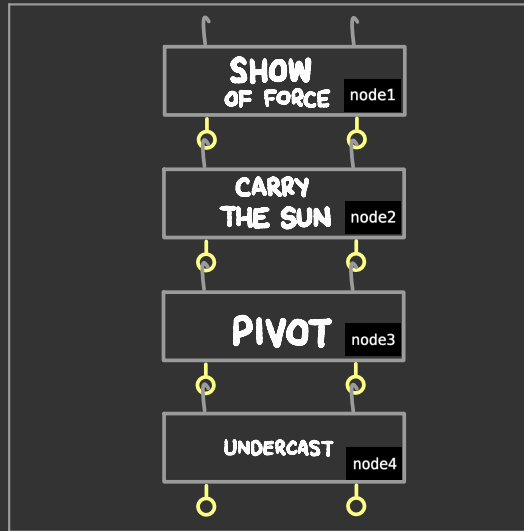
Code

a menu update at
bright ideas
brewing

```
In [6]: node1
Out[6]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['ABOVE THE ALPINE', None]]]]

In [7]: set_value(node4, "UNDERCAST")

In [8]: node1
Out[8]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['UNDERCAST', None]]]]
```



```
def make_node(x, y):
    | return [x, y]

def value(node):
    | return node[0]

def next(node):
    | return node[1]

def set_value(node, val):
    | node[0] = val

def set_next(node, next_node):
    | node[1] = next_node
```

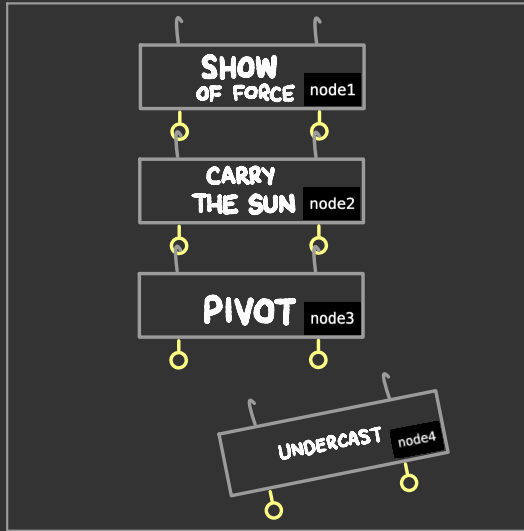
Code

a menu update at
bright ideas
brewing

```
In [8]: node1
Out[8]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', ['UNDERCAST', None]]]]

In [9]: set_next(node3, None)

In [10]: node1
Out[10]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', None]]]
```



```
def make_node(x, y):
    | return [x, y]

def value(node):
    | return node[0]

def next(node):
    | return node[1]

def set_value(node, val):
    | node[0] = val

def set_next(node, next_node):
    | node[1] = next_node
```

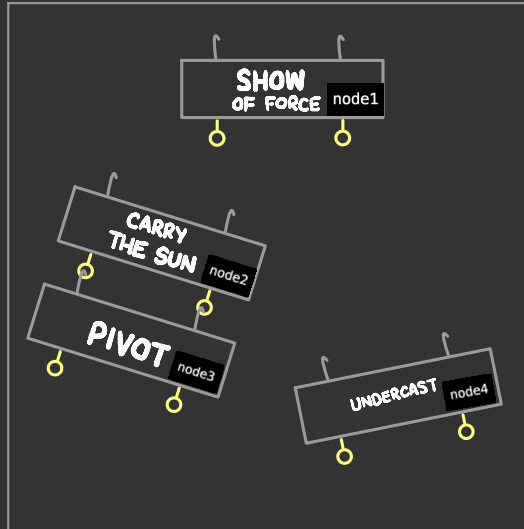
Code

a menu update at
bright ideas
brewing

```
In [10]: node1
Out[10]: ['SHOW OF FORCE', ['CARRY THE SUN', ['PIVOT', None]]]

In [11]: set_next(node1, None)

In [12]: node1
Out[12]: ['SHOW OF FORCE', None]
```



```
def make_node(x, y):
    return [x, y]

def value(node):
    return node[0]

def next(node):
    return node[1]

def set_value(node, val):
    node[0] = val

def set_next(node, next_node):
    node[1] = next_node
```

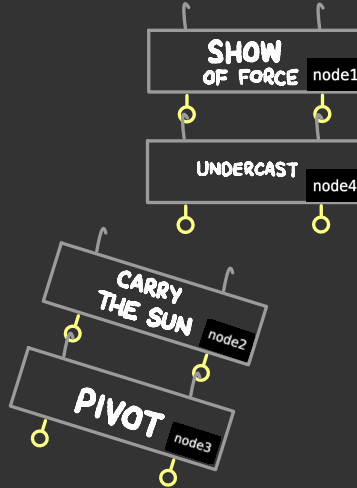
Code

a menu update at
bright ideas
brewing

```
In [12]: node1
Out[12]: ['SHOW OF FORCE', None]

In [13]: set_next(node1, node4)

In [14]: node1
Out[14]: ['SHOW OF FORCE', ['UNDERCAST', None]]
```



```
def make_node(x, y):
    | return [x, y]

def value(node):
    | return node[0]

def next(node):
    | return node[1]

def set_value(node, val):
    | node[0] = val

def set_next(node, next_node):
    | node[1] = next_node
```

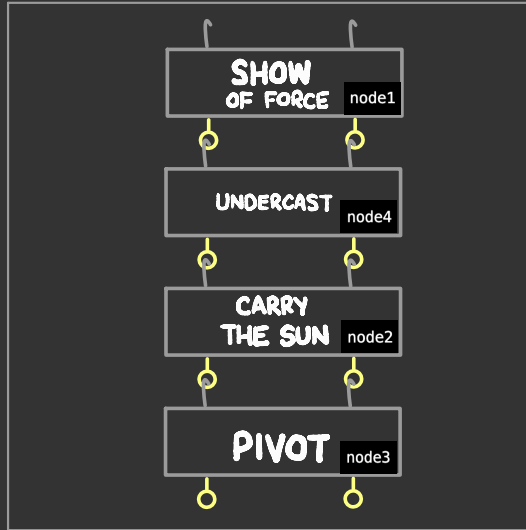
Code

a menu update at
bright ideas
brewing

```
In [14]: node1
Out[14]: ['SHOW OF FORCE', ['UNDERCAST', None]]

In [15]: set_next(node4, node2)

In [16]: node1
Out[16]: ['SHOW OF FORCE', ['UNDERCAST', ['CARRY THE SUN', ['PIVOT', None]]]]
```



```
def make_node(x, y):
|     return [x, y]

def value(node):
|     return node[0]

def next(node):
|     return node[1]

def set_value(node, val):
|     node[0] = val

def set_next(node, next_node):
|     node[1] = next_node
```

Code

this approach to organizing sequential data is called a **linked list**

